

Suivi du projet

Architecture

Interactions View - Controller

Pour respecter le MVC, la vue est responsable uniquement de l'affichage et ne contient pas de logique métier. Il existe un contrôleur spécifique pour chacune des fenêtres et la communication entre les fenêtres et leur contrôleur respectifs est assurée par une interface appelée `ViewListener`. Lors de chaque événement (comme appuyer un bouton), la vue ne traite pas directement l'action, mais délègue à son contrôleur via une méthode de `ViewListener`.

Les contrôleurs implémentent les interfaces respectives qui permettent de recevoir les événements issus des vues et de faire les actions nécessaires. Souvent, ces contrôleurs délèguent la logique à une couche supérieure (`ClientController`) via leur propre interface (`Listener`), ce qui permet de connecter les contrôleurs à un contrôleur central sans créer de dépendances fortes.

La logique métier et les interactions avec le serveur sont centralisées dans ce contrôleur principal, qui implémente les différentes interfaces `Listener` des contrôleurs de vue. `ClientController` s'occupe alors de changer de fenêtres et d'envoyer des messages au serveur pour transmettre ou obtenir les données nécessaires, qui sont ensuite transmises aux contrôleurs de vues pour pouvoir mettre à jour l'interface.

Cette structure permet une chaîne de communication claire : l'utilisateur interagit avec la vue, la vue notifie son contrôleur via l'interface `ViewListener`, le contrôleur transmet l'action au contrôleur central via l'interface `Listener`, puis les résultats sont renvoyés vers la vue pour l'affichage.

Structure Client - Server

Nous avons divisé notre application en deux parties, le client et le serveur. Cela nous permettra de faire une partie contre un autre joueur ainsi que de pouvoir communiquer via un Chat avec les autres joueurs.

Pour mettre en place cette nouvelle structure, nous avons déplacé le `Model`, `Service` et `Repository` dans le `Server` et laissé le reste (`View` et `Controller`) dans le client. La classe `ClientController` va alors faire des requêtes au serveur (recevoir une certaine donnée, choisir une action en combat, ...) et celui-ci va répondre à / appliquer la demande grâce au `Model` ou au `Service`. Etant donné que, précédemment, le contrôleur et la vue travaillaient directement avec des objets du Modèle (`Bugemon`, `Player`, `Ability`), nous les avons modifiés pour qu'ils prennent en compte des DTO, ne contenant que les données des objets.

La communication Client - Serveur se fait à l'aide de Socket et d'objets Sérialisables (objets qui peuvent être transmis dans un socket). Nous avons donc fait en sorte que tous les DTO implémentent Serializable afin qu'ils puissent être envoyés dans le socket.

Base de donnée

Pour la base de données, nous avons choisi d'utiliser SQLite en raison de sa simplicité, ainsi que de notre expérience préalable avec cet outil dans d'autres projets.

La base de données est composée de 14 tables. Nous avons créé 5 nouvelles classes Repository pour les manipuler.

Enfin, un script SQL a été mis en place pour initialiser les différentes tables de la base de données.

Social

Liste d'amis

Pour la gestion des amis, nous avons suivi la même architecture client-serveur que le reste de l'application. Le client envoie un message au serveur (SendFriendRequestMessage, AcceptFriendRequestMessage, DeclineFriendRequestMessage, GetFriendsListMessage, GetFriendRequestsMessage) et le serveur répond en manipulant la base de données via AccountService. Deux nouvelles tables ont été ajoutées à la base de données : Friends pour stocker les relations d'amitié, et friend_requests pour stocker les demandes en attente. L'interface sociale est accessible depuis la fenêtre principale via un bouton en haut à gauche de l'écran, qui ouvre un panneau permettant d'inviter un joueur, de consulter sa liste d'amis et de gérer les demandes reçues.

Chat

La fonctionnalité de chat s'appuie sur le même principe d'architecture client-serveur. Le client communique avec le serveur via l'envoi de messages (SendChatMessageMessage, GetChatMessagesMessage), lequel se charge de leur stockage et récupération en base de données par l'intermédiaire du ChatService. L'affichage s'effectue au sein du panneau social, avec un mécanisme de rafraîchissement automatique toutes les 3 secondes. Afin de garantir la fluidité de l'interface, ce processus est géré par des threads en arrière-plan. Enfin, pour améliorer la lisibilité de la conversation, les messages de l'utilisateur et ceux de son ami sont distingués par des couleurs différentes.

Refactor

Le refactor avait comme principal but d'améliorer la manière dont la Vue et le Controller discutent entre eux.

Nous avons aussi, enfin, retiré la classe `ServiceLoader` du code. Les services sont maintenant instanciés dans le `Main` et injectés dans le reste du code plutôt que stockés de façon statique. `ServiceLoader` servait un rôle de transition d'architecture et aurait dû être retirée en itération 2 mais nous avons à la place effectué d'autres tâches que nous avons jugées plus importantes.

Histoires inachevées

Histoire 25 : défier un autre joueur

L'histoire comprenait le fait de défier en 1 contre 1 un autre joueur dans un combat classique. Il fallait donc déplacer la logique de l'application vers la structure Client - Serveur. Cette restructuration ayant pris plus de temps que prévu, il n'a pas été possible d'implémenter le combat multijoueur par manque de temps.

Histoire 26 : Chat

Cette histoire était divisée en 2 parties : Le chat présent dans le menu principal et le chat présent pendant le combat (dans les deux cas, cela représentait une discussion privée avec un ami). Etant donné que le combat multijoueur n'a pas pu être terminé, seule la moitié de cette histoire (pouvoir discuter avec ses amis dans les menus) n'a pu être implémentée.